

SkiferHub

Guide de reference - AgenticHub

Interface conversationnelle pour vos pipelines Lakehouse

Vue d'ensemble

L'AgenticHub est le point d'entree unique de toutes les capacites agentiques de Skifer. Il route chaque question en langage naturel vers l'agent specialise competent, sans configuration requise.

5 agents integres | Routage sans LLM | Profil utilisateur persistant

Session avec TTL | CLI REPL riche | Export PDF de session

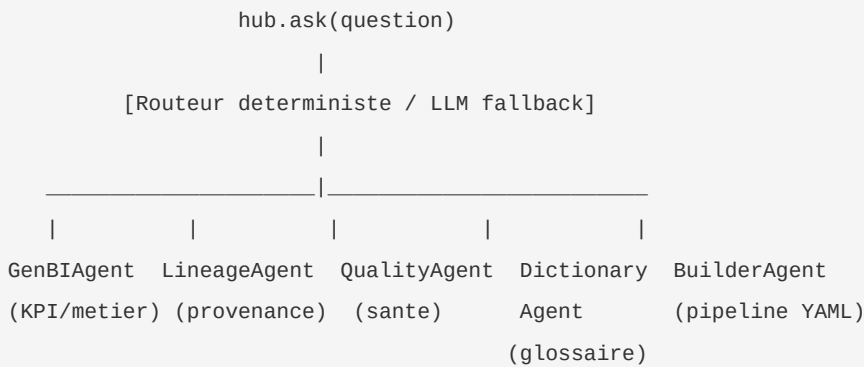
Table des matieres

1. Vue d'ensemble de l'AgenticHub
2. Demarrage : commande skifer hub
3. Routage des questions
4. Les 5 agents specialises
 - 4.1 GenBIAgent - requetes KPI / metier
 - 4.2 LineageAgent - provenance et impact
 - 4.3 QualityAgent - sante des donnees
 - 4.4 DictionaryAgent - glossaire metier
 - 4.5 BuilderAgent - generation de pipelines YAML
5. Historique de session (TTL + persistance)
6. Profil utilisateur et alias semantiques
7. Formats de reponse
8. Exemples de code Python
9. Reference rapide des mots-cles

1. Vue d'ensemble de l'AgenticHub

L'AgenticHub est un routeur conversationnel qui expose une API unifiée `hub.ask(question)` vers 5 agents spécialisés. Chaque question en langage naturel est analysée par mots-clés (déterministe, sans LLM) puis dirigée vers l'agent le plus pertinent.

Architecture



Dependances minimales

Tous les agents sont OPTIONNELS. Si l'agent cible n'est pas injecté, `hub.ask()` retourne un `AgentResponse(mode='error')` avec un message explicite. Aucun Spark ni LLM n'est requis pour le routage de base.

Constructeur AgenticHub

```
AgenticHub(
    semantic_engine=None,      # SemanticEngine (requis pour GenBIAgent auto)
    llm_provider=None,        # LLMProvider (optionnel, active fallback LLM)
    lineage_agent=None,       # LineageAgent injecte directement
    quality_agent=None,      # QualityAgent injecte directement
    dictionary_agent=None,   # DictionaryAgent injecte directement
    builder_agent=None,      # BuilderAgent injecte directement
    genbi_agent=None,        # GenBIAgent injecte directement
    session_title='SkiiferHub',
    profile=None,             # UserProfile (alias semantiques)
)
```

2. Demarrage : commande skifer hub

Installation

```
pip install skifer
# ou en mode dev :
pip install -e '[dev]'
```

Lancement

```
# Config du repertoire courant
skifer hub

# Config explicite
skifer hub --config path/vers/config.yaml

# Forcer une nouvelle session (efface la precedente)
skifer hub --new-session
```

Demarrage de session

Au lancement, si une session non expiree existe pour le scope (catalog + env), le hub propose deux options :

```
SkiferHub v0.9.0 -- catalog: my_catalog (DEV)
-----
[1] Nouvelle session (efface la session precedente)
[2] Reprendre la session du 2026-05-01 (11 messages)

Choix [1/2] :
```

Commandes speciales dans le REPL

Commande	Action
exit / quit	Quitter le REPL proprement
help	Afficher la liste des commandes
history	Afficher les 5 dernieres questions de la
export pdf	Exporter la session en PDF (fpdf2 requis)

Scope de session

Chaque session est liee a un scope derive de (catalog, env) defini dans config.yaml. Deux projets avec des catalogs differents ont des sessions independantes. TTL par default : 15 jours. Les sessions expirees sont purgees automatiquement au demarrage.

3. Routage des questions

Principe : deterministe d'abord, LLM en fallback

Le hub analyse la question en 3 etapes : (1) resolution des alias utilisateur, (2) detection d'intent par mots-cles normalises (minuscules, sans accents), (3) si aucun mot-cle ne matche et qu'un LLM est disponible, un seul appel LLM minimal classifie l'intent parmi les 5 categories.

Table de routage par mots-cles

Intent detecte	Mots-cles reconnus (exemples)	Agent cible
lineage	d'ou, vient, provenance, upstream, origi	LineageAgent
lineage	downstream, affecte, depend, come from	LineageAgent
dictionary	que signifie, definition, glossaire, voc	DictionaryAgent
quality	qualite, saine, anomalie, checks, doublo	QualityAgent
builder	cree, genere, construis, yaml, pipeline,	BuilderAgent
genbi	(aucun mot-cle specifique reconnu) --	GenBIAgent

Normalisation des textes

Avant la comparaison, chaque question est normalisee : minuscules + suppression des accents (NFD). Ainsi 'D'ou vient ?' et 'd'ou vient ?' et 'd ou vient ?' sont tous traites de la meme facon.

Fallback LLM (optionnel)

Si llm_provider est fourni et qu'aucun mot-cle n'a matche, un prompt minimal est envoye : question + liste des 5 intents possibles. La reponse attendue est un seul mot parmi : genbi, lineage, quality, dictionary, builder. Si la reponse LLM est invalide ou si l'appel echoue, GenBIAgent est utilise en fallback ultime.

4. Les 5 agents specialises

4.1 GenBIAgent -- requetes KPI / metier

Agent par default (fallback). Traduit les questions metier en requetes semantiques, les execute via Spark et retourne un resultat formate. Necessite un SemanticEngine et un LLMProvider.

Exemple de questions reconnues :

- Quel est le CA par region ce trimestre ?
- Montre-moi les ventes de juin 2026
- Calcule le taux de transformation par canal
- Compare le revenu brut entre EMEA et APAC

Constructeur :

```
GenBIAgent(  
    semantic_engine,          # SemanticEngine (requis)  
    llm_provider,             # LLMProvider (requis)  
    history=False,            # activer le log SessionHistory  
    session_title='Analyse KPI'  
)
```

Retourne : AgentResponse

4.2 LineageAgent -- provenance et impact

Analyse le lignage colonne-niveau. Consomme le LineageTracker et le DataDictionary. Aucun Spark ni LLM requis pour le coeur fonctionnel.

Modes disponibles :

Mode	Description	Exemple
trace	Upstream : d'ou vient ce champ ?	D'ou vient amount_eur ?
impact	Downstream : quelles cols dependent ?	Quel impact si on modifie order_id ?
render	Visualisation du graphe complet	Affiche le diagramme de lineage
lookup	Lookup DataDictionary (definition)	Que signifie gross_revenue ?

Constructeur :

```
LineageAgent(  
    schema_dict=None,      # dict schema SkiFerEngine (optionnel)  
    llm_provider=None,     # LLMPProvider (optionnel -- pour narration)  
    history=False,         # log dans SessionHistory  
)
```

Retourne : LineageResponse

4.3 QualityAgent -- sante des donnees

Execute des checks de qualite sur les tables via le DataMonitor. Un backend (objet avec methode sql()) est requis pour executer les checks. LLM optionnel pour les resumes narratifs.

Modes disponibles :

Mode	Description	Exemple
check	Executer les checks sur une table	La table fact_orders est-elle saine ?
history	Historique des derniers checks	Evolution de la qualite de orders ?
report	Generer un rapport formate text/json/htm	Genere le rapport de qualite orders

Constructeur :

```
QualityAgent(  
    backend,                # Backend (requis pour les checks live)  
    history_store=None,     # HistoryStore (optionnel)  
    llm_provider=None,     # LLMPProvider (optionnel)  
)
```

Retourne : QualityResponse

4.4 DictionaryAgent -- glossaire metier

Fournit les definitions, types et meta-attributs des champs depuis le DataDictionary. Peut enrichir les reponses avec le GlossaryReader. Aucun Spark requis.

Modes disponibles :

Mode	Description	Exemple
lookup	Definition d'un champ specifique	Que signifie gross_revenue ?
list	Lister les champs d'une table	Liste les colonnes de fact_orders
export	Exporter le dictionnaire complet en JSON	Exporte le dictionnaire

Constructeur :

```
DictionaryAgent(  
    schema_dict=None,      # dict schema (optionnel)  
    llm_provider=None,     # LLMPProvider (optionnel)  
    session_history=False, # log dans SessionHistory  
)
```

Retourne : DictionaryResponse

4.5 BuilderAgent -- generation de pipelines YAML

Genere des schemas YAML pour SkiferEngine depuis une description en langage naturel (mode LLM) ou un assistant interactif (wizard). Distinct du SemanticBuilder qui genere des modeles semantiques.

Deux modes :

Mode	Description	LLM requis ?
ask()	Description NL -> JSON structure -> YAML	Oui
wizard()	Assistant pas-a-pas en ligne de commande	Non

Exemple de dialogue hub/builder :

```
hub.ask("cree un pipeline qui joint orders et customers
        sur customer_id, filtre region EMEA")

# BuilderAgent genere :
# tables:
#   - name: catalog.silver.orders
#     alias: ord
#     filter: [region>equals:EMEA]
#   - name: catalog.silver.customers
#     alias: cust
# join:
#   - table_from: [ord, customer_id]
#     table_to: [cust, id]
#     type: left
```

Constructeur :

```
BuilderAgent(
    backend,          # Backend (requis)
    catalog=None,     # prefixe catalogue
    llm_provider=None # LLMPProvider (requis pour mode ask())
)
```

Retourne : BuilderResponse

Note : BuilderAgent.ask() a une signature differente des autres agents :

```
# Tous les autres agents :
agent.ask(question: str) -> AgentResponse|...

# BuilderAgent uniquement :
builder.ask(description: str, output_dir: str = 'schemas') -> BuilderResponse
```

5. Historique de session (TTL + persistance)

Chaque session AgentiHub est loguee dans un SessionHistory unique. Les sessions sont persistees sur disque et chargees automatiquement au prochain lancement pour le meme scope.

Localisation des fichiers

```
~/skifer_sessions/<scope_id>.json

scope_id = f'{catalog}_{env}' # derive de config.yaml
# Exemple : my_catalog_DEV.json
```

Attributs de SessionHistory

Attribut	Type	Description
title	str	Titre de la session
session_id	str	UUID4 genere automatiquement
created_at	datetime	Date/heure de creation UTC
ttd_days	int	Duree de vie en jours (defaut : 15)
is_expired	bool	True si TTL depasse

Methodes cles

Methode	Description
add(entry)	Ajouter une entree HistoryEntry
entries()	Retourner la liste des entrees (copie)
save(scope_id)	Persister dans ~/skifer_sessions/
load_or_create(scope_id)	Charger ou creer une nouvelle session
purge_expired(scope_id)	Supprimer les sessions expirees
to_json(path)	Exporter en JSON
from_json(path)	Recharger depuis un fichier JSON
to_pdf(path)	Exporter en PDF (fpdf2 requis)

Exemple d'utilisation directe

```
from skifer.agentic.history import SessionHistory

# Charger ou creer une session
session = SessionHistory.load_or_create('my_catalog_DEV')

# Verifier le TTL
if session.is_expired:
    session = SessionHistory() # nouvelle session
```


Sauvegarder apres chaque interaction

```
session.save('my_catalog_DEV')
```


Exporter

```
session.to_pdf('rapport_session.pdf')
```

```
session.to_json('session_backup.json')
```

6. Profil utilisateur et alias sémantiques

Le UserProfile permet de définir des alias personnels qui sont résolus automatiquement avant chaque routage. Il mémorise aussi les filtres par défaut et le format de sortie préféré.

Fichier de profil

```
# ~/.skiher_profile.yaml
version: 1
aliases:
  ventes: catalog.silver.orders
  ca: amount_eur
default_filters:
  - region>equals:EMEA
preferred_format: table
blacklist_suggestions:
  - env>equals:PROD
```

Exemple de résolution d'alias

```
from skiher.agentic.user_profile import UserProfile
from skiher.agentic.hub import AgenticHub

profile = UserProfile.load() # charge ~/.skiher_profile.yaml
# Ou créer manuellement :
profile = UserProfile(aliases={'ventes': 'catalog.silver.orders'})

hub = AgenticHub(lineage_agent=..., profile=profile)

# Question avec alias :
hub.ask("d'où vient ventes ?")
# Avant routage, 'ventes' est remplacé par 'catalog.silver.orders'
# -> équivalent à : "d'où vient catalog.silver.orders ?"
```

Méthodes de gestion des alias

Méthode	Description	
resolve_alias(term)	Retourner le nom canonique ou None	
add_alias(term, canonical)	Ajouter et sauvegarder un alias	
reject_suggestion(term)	Blacklister une suggestion	
save(path)	Sauvegarder le profil en YAML	
UserProfile.load(path)	Charger le profil (vide si absent)	

Normalisation des alias

Les clés d'alias sont normalisées automatiquement (minuscules + suppression des accents). Ainsi 'CA', 'Ca', 'ca' et 'CA' sont tous le même alias. La résolution utilise des frontières de mots (\b) pour éviter les remplacements de sous-chaînes (ex : 'ca' ne modifie pas 'cascade').

7. Formats de reponse

Chaque agent retourne un dataclass structure specifique. Le CLI les affiche automatiquement. En Python, vous pouvez les inspecter directement.

AgentResponse (GenBIAgent)

Champ	Type	Description
question	str	Question originale
mode	str	query view ambiguous error
model_used	str	Cle du modele semantique selectionne
explanation	str	Ce que l'agent a compris
result	Any	FormattedResult (query) ou FQN vue (view)
error	str None	Message d'erreur si mode=error
clarification_question	str None	Question si mode=ambiguous
success	bool	True si mode=query ou view, sans erreur

LineageResponse (LineageAgent)

Champ	Description
question	Question originale
mode	trace impact render lookup
table	Table ciblee
column	Colonne ciblee
edges	list[LineageEdge] - aretes du graphe
diagram	Diagramme Mermaid / JSON / HTML (mode re
narrative	Explication en prose (LLM optionnel)
field_entry	FieldEntry du DataDictionary (mode looku
error	Message d'erreur ou None

QualityResponse (QualityAgent)

Champ	Description
mode	check history report
table	FQN de la table ciblee
report	MonitorReport du dernier check
history_reports	list[MonitorReport] (mode history)
text_output	Resultat formate (text/json/html)
passed	bool None - True si aucun echec critiq

DictionaryResponse (DictionaryAgent)

Champ	Description	
mode	lookup list export	
table	Table ciblée	
column	Colonne ciblée	
entry	FieldEntry trouvée (mode lookup)	
entries	list[FieldEntry] (mode list)	
text_output	Dictionnaire formaté (mode export)	
suggestions	Noms proches si champ non trouvé	

BuilderResponse (BuilderAgent)

Champ	Description	
success	True si YAML produit et valide	
yaml_content	Contenu YAML du pipeline généré	
output_path	Chemin du fichier sauvegarde ou None	
error	Message d'erreur si success=False	
clarification_question	Question si une table/col est ambiguë	

8. Exemples de code Python

Usage minimal (sans Spark ni LLM)

```
from skifer.agentic.hub import AgenticHub
from skifer.agentic.lineage_agent import LineageAgent
from skifer.agentic.models import LineageResponse

lineage = LineageAgent(schema_dict=my_core_schema)
hub = AgenticHub(lineage_agent=lineage)

resp = hub.ask("D'ou vient le champ amount_eur ?")
assert isinstance(resp, LineageResponse)
print(resp.narrative)
```

Hub complet avec tous les agents

```
from skifer.agentic.hub import AgenticHub
from skifer.agentic.lineage_agent import LineageAgent
from skifer.agentic.quality_agent import QualityAgent
from skifer.agentic.dictionary_agent import DictionaryAgent
from skifer.agentic.builder_agent import BuilderAgent
from skifer.agentic.user_profile import UserProfile

hub = AgenticHub(
    semantic_engine=semantic_engine,
    llm_provider=llm_provider,
    lineage_agent=LineageAgent(schema_dict=schema),
    quality_agent=QualityAgent(backend=backend),
    dictionary_agent=DictionaryAgent(schema_dict=schema),
    builder_agent=BuilderAgent(backend=backend, llm_provider=llm),
    profile=UserProfile.load(),
)
```

Gestion de la session persistante

```
from skifer.agentic.history import SessionHistory

scope = 'my_catalog_DEV'
SessionHistory.purge_expired(scope)
session = SessionHistory.load_or_create(scope)

hub._history = session # injecter la session dans le hub

resp = hub.ask('quel est le CA ?')
session.save(scope)    # persister apres chaque interaction
```

Profil avec alias

```
from skifer.agentic.user_profile import UserProfile

profile = UserProfile()
profile.add_alias('ventes', 'catalog.silver.orders')
profile.add_alias('CA', 'amount_eur')

hub = AgenticHub(lineage_agent=lineage, profile=profile)

# Maintenant on peut dire :
hub.ask("d'ou vient le CA dans ventes ?")
# Resolu en : "d'ou vient le amount_eur dans catalog.silver.orders ?"
```

Inspecter la reponse selon le type

```
from skifer.agentic.models import (
    AgentResponse, LineageResponse, QualityResponse,
    DictionaryResponse, BuilderResponse
)

resp = hub.ask(question)

if isinstance(resp, AgentResponse):
    if resp.success:
        print(resp.result)
    elif resp.mode == 'error':
        print('Erreur :', resp.error)

elif isinstance(resp, LineageResponse):
    print('Edges :', resp.edges)
    if resp.diagram:
        print(resp.diagram)  # Mermaid

elif isinstance(resp, QualityResponse):
    print('Passe :', resp.passed)
    print(resp.text_output)

elif isinstance(resp, BuilderResponse):
    if resp.success:
        print(resp.yaml_content)
```

9. Reference rapide des mots-cles

Mots-cles de routage

LineageAgent

d'ou, d'ou, vient, provenance, upstream, origine
source de, where does, come from, provient
impact, downstream, affecte, utilise par, depend
depends on, affected

DictionaryAgent

que signifie, definis, definition, glossaire
vocabulaire, what is, what does mean, signification

QualityAgent

qualite, saine, anomalie, checks, nulls
doublons, health, monitor

BuilderAgent

cree, genere, construis, yaml, pipeline
schema, build, create

GenBIAgent

(aucun mot-cle specifique -- fallback)
Exemples : quel est, montre-moi, calcule, compare

Signatures de reference

```
# Tous les agents (sauf BuilderAgent) :
agent.ask(question: str) -> AgentResponse | LineageResponse | ...

# BuilderAgent specifique :
builder.ask(description: str, output_dir: str = 'schemas') -> BuilderResponse
builder.wizard() -> str    # REPL interactif -- ne pas appeler depuis hub.ask()

# Hub :
hub.ask(question: str, **kwargs) -> HubResponse
hub.history -> SessionHistory
```

Imports essentiels

```
from skifer.agentic import (
    AgenticHub, GenBIAgent, LineageAgent, QualityAgent,
```

DictionaryAgent, BuilderAgent,

AgentResponse, LineageResponse, QualityResponse,

DictionaryResponse, BuilderResponse,

SessionHistory, HistoryExporter, UserProfile,

Skifer -- skifer.cli:main | pip install skifer